UNIVERSIDADE TIRADENTES
DIREÇÃO DE GRADUAÇÃO
SISTEMAS DE INFORMAÇÃO / ADS / CIÊNCIAS DA COMPUTAÇÃO

ENZO LEONARDO REIS SANTANA
OTÁVIO HENRIQUE SANTOS MERCENA
WAGNER RODRIGUES CHAGAS

CRIPTOGRAFIA ASSIMÉTRICA E CONFIANÇA

cifragem híbrida (AES-GCM e RSA-OAEP) e assinatura digital (Ed25519)

ARACAJU
2026

ENZO LEONARDO REIS SANTANA
OTÁVIO HENRIQUE SANTOS MERCENA
WAGNER RODRIGUES CHAGAS

CRIPTOGRAFIA ASSIMÉTRICA E CONFIANÇA

cifragem híbrida (AES-GCM e RSA-OAEP) e assinatura digital (Ed25519)

Documentação técnica da Entrega 1 do Projeto Integrador "Segurança Aplicada ao Desenvolvimento", apresentada à disciplina Segurança da Informação do curso de Sistemas de Informação/ADS da Universidade Tiradentes, como atividade avaliativa.

Docente: Profa. Ana Carla do Nascimento Santos

ARACAJU
2026

SUMÁRIO

1 PROBLEMA E OBJETIVO 4
2 AMBIENTE E TECNOLOGIAS 4
3 ALGORITMOS, BIBLIOTECAS E JUSTIFICATIVA 4
3.1 Cifragem híbrida 5
3.2 Assinatura digital 5
3.3 Sigilo e autenticidade 5
3.4 Certificados digitais e PKI 6
3.5 Gestão de chaves 6
4 IMPLEMENTAÇÃO E INSTRUÇÕES DE EXECUÇÃO 6
4.1 Estrutura do repositório 6
4.2 Fluxo da solução 6
4.3 Instruções de execução 7
5 TESTES E EVIDÊNCIAS 7
6 RISCOS, LIMITAÇÕES E MELHORIAS 8
REFERÊNCIAS 9
APÊNDICE A – SAÍDA DA EXECUÇÃO 10

1 PROBLEMA E OBJETIVO

Aplicações modernas trocam mensagens entre serviços por redes que não podem ser consideradas confiáveis. Nesse cenário, um serviço remetente (A) precisa enviar uma mensagem a um serviço destinatário (B) de modo que apenas B consiga lê-la e que B possa comprovar que ela foi produzida por A e não sofreu alteração durante o trânsito.

O objetivo desta entrega é implementar, testar e justificar uma solução que atenda a esses dois requisitos usando criptografia assimétrica. O enunciado admite cifragem híbrida ou assinatura digital; a solução desenvolvida implementa as duas, combinadas, de forma que o sigilo e a autenticidade sejam tratados como propriedades distintas e verificadas separadamente. O trabalho também explica a diferença entre sigilo e autenticidade e descreve o papel de certificados digitais e da infraestrutura de chaves públicas (PKI).

2 AMBIENTE E TECNOLOGIAS

O código foi desenvolvido em Python 3.10 ou superior, linguagem escolhida pela clareza e pela disponibilidade de bibliotecas criptográficas maduras. A única dependência é a biblioteca cryptography, mantida pela Python Cryptographic Authority, que expõe primitivas consolidadas e utiliza o OpenSSL como motor (PYCA, 2026). Conforme exigido na atividade, nenhum algoritmo criptográfico foi implementado de forma própria.

A execução foi feita em ambiente local, com ambiente virtual (venv) e dependências listadas no arquivo requirements.txt, o que permite reproduzir o experimento em qualquer máquina com Python. Todos os dados utilizados são fictícios.

3 ALGORITMOS, BIBLIOTECAS E JUSTIFICATIVA

A Tabela 1 resume as escolhas e os parâmetros adotados.

Tabela 1 – Algoritmos e parâmetros utilizados

Função	**Algoritmo**	**Parâmetros**	**Justificativa**
Cifrar o conteúdo	AES-GCM	Chave de 256 bits; nonce de 96 bits aleatório; tag de 128 bits; AAD com cabeçalho	Cifra autenticada (AEAD), com sigilo e integridade, eficiente para dados grandes
Encapsular a chave simétrica	RSA-OAEP	RSA de 3072 bits; expoente 65537; SHA-256; MGF1 com SHA-256	Preenchimento seguro para RSA; 3072 bits equivale a cerca de 128 bits de segurança

Função	Algoritmo	Parâmetros	Justificativa
Assinar o pacote	Ed25519	Chave de 256 bits	Assinatura determinística, rápida e sem parâmetros inseguros a configurar

Fonte: elaborado pelos autores (2026).

3.1 Cifragem híbrida

O RSA é computacionalmente custoso e só cifra mensagens menores que o seu módulo, o que o torna inadequado para cifrar conteúdos extensos. A cifragem simétrica, por outro lado, é rápida, mas exige que as partes compartilhem previamente uma chave. A cifragem híbrida combina as duas abordagens: uma chave AES-256 efêmera, gerada para cada mensagem, cifra o conteúdo, e essa chave é protegida com a chave pública RSA do destinatário, usando o esquema OAEP (MORIARTY et al., 2016).

O modo GCM é uma cifra autenticada: além de cifrar, produz uma etiqueta de autenticação que permite detectar qualquer modificação no texto cifrado ou nos dados associados (DWORKIN, 2007). Um requisito crítico do GCM é jamais repetir um nonce com a mesma chave; por isso o nonce é sorteado a cada mensagem e a chave AES nunca é reutilizada nem gravada em disco.

3.2 Assinatura digital

A assinatura digital utiliza o par de chaves do remetente: A assina com sua chave privada e qualquer interessado verifica com a chave pública correspondente. Escolheu-se o Ed25519, uma instância do EdDSA descrita na RFC 8032 (JOSEFSSON; LIUSVAARA, 2017), por ser determinístico, o que elimina falhas associadas à geração ruim de números aleatórios por assinatura, e por não exigir escolhas de parâmetros que possam ser configuradas de forma insegura.

Adotou-se a ordem “cifrar e depois assinar” (encrypt-then-sign): a assinatura cobre o pacote já cifrado (nonce, chave encapsulada, texto cifrado e dados associados), serializado de forma determinística. Assim, o destinatário verifica a assinatura antes de qualquer operação de decifragem, rejeitando pacotes adulterados ou de origem desconhecida sem processá-los.

3.3 Sigilo e autenticidade

Sigilo (confidencialidade) é a propriedade que impede que terceiros leiam o conteúdo. Na solução, é obtido pelo AES-GCM combinado com o RSA-OAEP: como a chave simétrica

é encapsulada com a chave pública de B, somente o detentor da chave privada de B consegue recuperá-la e decifrar a mensagem.

Autenticidade é a propriedade que permite confirmar quem produziu a mensagem e que ela não foi alterada (integridade), o que sustenta também o não repúdio. É obtida pela assinatura digital: apenas o detentor da chave privada de A consegue produzir uma assinatura que a chave pública de A valide.

As duas propriedades são independentes. Qualquer pessoa que conheça a chave pública de B pode cifrar uma mensagem para ele, e a mensagem será sigilosa, porém sem nenhuma garantia de origem. Em sentido oposto, um documento assinado e transmitido em claro é autêntico, mas legível por todos. Por isso a solução aplica ambos os mecanismos.

3.4 Certificados digitais e PKI

Uma chave pública isolada não prova a quem pertence. Um atacante em posição intermediária poderia substituir a chave pública de B pela sua própria e, com isso, ler e reenviar as mensagens (ataque do homem no meio). Um certificado digital X.509 resolve esse problema ao vincular uma identidade (pessoa, domínio ou serviço) a uma chave pública, e é assinado por uma Autoridade Certificadora (AC) (COOPER et al., 2008).

A infraestrutura de chaves públicas (PKI) é o conjunto de ACs, cadeias de confiança (da raiz às intermediárias e ao certificado final), políticas, prazos de validade e mecanismos de revogação, como CRL e OCSP, que permite a terceiros confiar nessas ligações entre identidade e chave. No Brasil, a ICP-Brasil é um exemplo de PKI. No laboratório desta entrega, as chaves públicas são trocadas diretamente entre os serviços; em produção, as chaves de A e de B seriam distribuídas em certificados validados por uma AC confiável.

3.5 Gestão de chaves

- As chaves privadas são salvas em PEM/PKCS#8 cifradas por senha e com permissão restrita ao proprietário do arquivo (chmod 600).
- O diretório keys/ e os arquivos .pem estão no .gitignore, de modo que nenhuma chave privada seja publicada no repositório.
- A senha usada no código é apenas de laboratório; em produção, o segredo deve ficar em um cofre (por exemplo, KMS ou Vault).
- A chave AES é efêmera: uma por mensagem, nunca persistida.

– Recomenda-se rotação periódica das chaves de longa duração e revogação imediata em caso de comprometimento (BARKER, 2020).

4 IMPLEMENTAÇÃO E INSTRUÇÕES DE EXECUÇÃO

4.1 Estrutura do repositório

A entrega está na pasta entrega1-cripto do repositório do projeto e contém o código-fonte (entrega1_cripto.py), o README e a pasta evidencias, com a saída da execução e um pacote protegido de exemplo.

4.2 Fluxo da solução

O fluxo implementado segue as etapas abaixo.

1) O remetente gera uma chave AES-256 e um nonce de 96 bits aleatórios.
2) A mensagem é cifrada com AES-GCM, usando um cabeçalho como dado autenticado associado.
3) A chave AES é encapsulada com a chave pública RSA do destinatário (OAEP com SHA-256).
4) O remetente assina com Ed25519 a serialização determinística do pacote.
5) O destinatário verifica a assinatura antes de decifrar; em seguida recupera a chave AES com sua chave privada e decifra a mensagem, validando a etiqueta do GCM.

O trecho a seguir mostra o núcleo da operação de proteção:

```
chave_aes = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)
texto_cifrado = AESGCM(chave_aes).encrypt(nonce, mensagem,
cabecalho)
chave_encapsulada = rsa_pub.encrypt(chave_aes, oaep())
pacote["assinatura"] = b64e(ed_priv.sign(serializar(pacote)))
```

Fonte: elaborado pelos autores (2026).

4.3 Instruções de execução

Os comandos a seguir reproduzem o experimento a partir da raiz do repositório:

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cd entrega1-cripto
python entrega1_cripto.py
```

A execução gera o diretório keys/ (não versionado) e o arquivo evidencias/pacote_exemplo.json.

5 TESTES E EVIDÊNCIAS

Foram executados cinco cenários, resumidos na Tabela 2. A saída completa da execução consta no Apêndice A e no arquivo evidencias/saida_execucao.txt do repositório.

Tabela 2 – Cenários de teste e resultados

Nº	Cenário	Resultado esperado	Resultado obtido
1	Pacote íntegro	Mensagem original recuperada	Conforme esperado
2	Um bit do texto cifrado alterado	Assinatura inválida; pacote rejeitado	Conforme esperado
3	Atacante altera o pacote e o assina com outra chave	Assinatura inválida	Conforme esperado
4	Assinatura ignorada, texto cifrado adulterado	Etiqueta GCM inválida (defesa em profundidade)	Conforme esperado
5	Destinatário errado tenta decifrar	Decifragem rejeitada	Conforme esperado

Fonte: elaborado pelos autores (2026).

Os testes 2 e 3 demonstram a falha após adulteração: qualquer alteração, mesmo de um único bit, invalida a assinatura, e um atacante sem a chave privada do remetente não consegue produzir uma assinatura válida. O teste 4 mostra que, mesmo sem a assinatura, o AES-GCM detecta a modificação, caracterizando defesa em profundidade. O teste 5 confirma o sigilo: sem a chave privada correta, a chave simétrica não é recuperada.

Além dos testes escritos, a evidência de execução em vídeo acompanha a entrega: [INSERIR LINK DO VÍDEO].

6 RISCOS, LIMITAÇÕES E MELHORIAS

- Ausência de PKI real: não há validação da identidade dona das chaves públicas. Melhoria: gerar certificados X.509 e validar a cadeia de confiança.
- Ausência de proteção contra replay: um pacote válido pode ser reenviado. Melhoria: incluir identificador único e carimbo de tempo dentro da parte assinada.
- Ausência de sigilo perfeito futuro: o comprometimento da chave RSA do destinatário expõe mensagens antigas interceptadas. Melhoria: usar troca de chaves efêmera (X25519) com HKDF.
- Chaves em disco com senha fixa no código, aceitável apenas em laboratório. Melhoria: usar gerenciador de segredos.

– RSA e Ed25519 não resistem a computadores quânticos de grande escala. Melhoria: acompanhar a migração para algoritmos pós-quânticos padronizados pelo NIST (ML-KEM e ML-DSA).

REFERÊNCIAS

BARKER, E. *Recommendation for key management: part 1 – general*. Gaithersburg: NIST, 2020. (NIST Special Publication 800-57 Part 1 Rev. 5). Disponível em: https://doi.org/10.6028/NIST.SP.800-57pt1r5. Acesso em: 8 nov. 2026.

COOPER, D. et al. *Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile*. [S. l.]: IETF, 2008. (RFC 5280). Disponível em: https://www.rfc-editor.org/rfc/rfc5280. Acesso em: 8 nov. 2026.

DWORKIN, M. *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC*. Gaithersburg: NIST, 2007. (NIST Special Publication 800-38D). Disponível em: https://doi.org/10.6028/NIST.SP.800-38D. Acesso em: 8 nov. 2026.

JOSEFSSON, S.; LIUSVAARA, I. *Edwards-curve digital signature algorithm (EdDSA)*. [S. l.]: IETF, 2017. (RFC 8032). Disponível em: https://www.rfc-editor.org/rfc/rfc8032. Acesso em: 8 nov. 2026.

MORIARTY, K. et al. *PKCS #1: RSA cryptography specifications version 2.2*. [S. l.]: IETF, 2016. (RFC 8017). Disponível em: https://www.rfc-editor.org/rfc/rfc8017. Acesso em: 8 nov. 2026.

OWASP. *Cryptographic storage cheat sheet*. [S. l.]: OWASP Foundation, [2026]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html. Acesso em: 8 nov. 2026.

PYTHON CRYPTOGRAPHIC AUTHORITY (PYCA). *Cryptography documentation*. [S. l.]: PyCA, 2026. Disponível em: https://cryptography.io. Acesso em: 8 nov. 2026.

APÊNDICE A – SAÍDA DA EXECUÇÃO

```
=== ENTREGA 1: Criptografia assimétrica e confiança ===

[1] Chaves geradas (RSA-3072 e Ed25519) e salvas em ./keys/
[2] Mensagem original: Transferir R$ 1.000,00 para a conta 1234-5
[3] Pacote protegido (texto cifrado, base64):
IiFuH7Qhh0I9x8JlrlhJh9Aizh7q2xlugq2Dm0/d...

--- TESTE 1: pacote íntegro ---
Sucesso. Mensagem recuperada: Transferir R$ 1.000,00 para a conta 1234-5

--- TESTE 2: 1 bit do texto cifrado adulterado ---
FALHA: assinatura inválida (origem/integridade não confirmadas)

--- TESTE 3: atacante re-assina com a própria chave ---
FALHA: assinatura inválida (origem/integridade não confirmadas)

--- TESTE 4: assinatura ignorada: o AES-GCM também protege ---
FALHA: tag GCM inválida (o próprio AES-GCM detecta a adulteração)

--- TESTE 5: destinatário errado tenta decifrar ---
FALHA: decifragem rejeitada (dados adulterados ou chave errada)

Todos os testes concluídos como esperado.
```